

knoad

WALLET

A wallet that's always one step ahead.

Fresh signing keys. Persistent ownership. A self-custodial design for a changing threat landscape.

CRONOSYNTHESIS

Technical white paper · 21 October 2026
Based on the 16 October 2026 implementation paper. Spec 1.4 remains normative.

PRE-AUDIT — NOT CLEARED FOR ASSETS OF VALUE

Part I — The idea

WHAT IF YOUR WALLET WAS ALWAYS ONE STEP AHEAD?

Imagine a password that changed itself after every use. Even if someone discovered the old one, it could no longer open the door.

That's the idea behind KNOAD.

Unlike a traditional wallet that keeps relying on the same spending key, KNOAD automatically replaces its active spending key after each included, successfully validated wallet operation. Yesterday's key cannot authorize tomorrow's transactions on that chain.

Keeping yesterday's access out of tomorrow's hands.
If you know, you KNOAD.

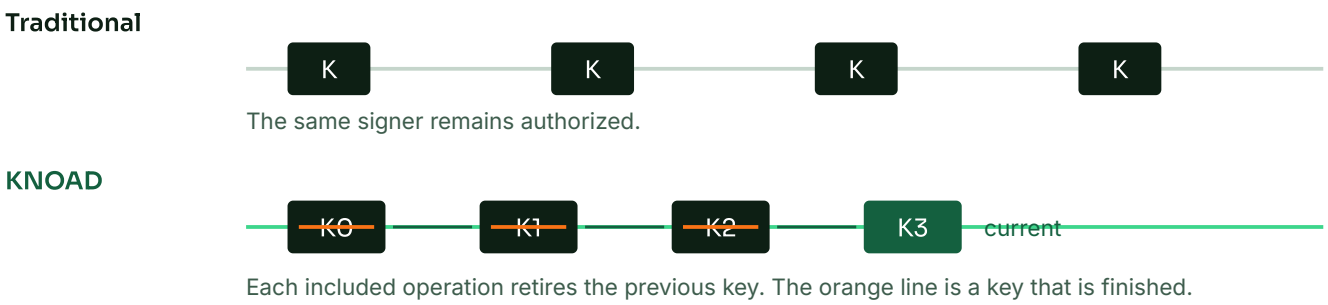
The short version

Your crypto wallet is protected by a digital key. In a traditional self-custodial wallet, that spending key usually remains authorized for the life of the address. If someone copies it, the key continues to work until the assets are moved or some other intervention changes control.

KNOAD is built around a different idea: your address should be durable, but the key allowed to spend should not be. After the network includes a qualifying operation, the wallet installs a fresh spending key and stops accepting the previous one. You don't have to generate, remember, or manually swap keys. Your assets do not need to move to a new address just because the daily spending key changes.

KNOAD is self-custodial. A separate 24-word recovery phrase belongs to you, not to Cronosynthesis, and is distinct from the spending key that changes. That separation is central to the design.

WHY IT MATTERS / KEY ACCESS OVER TIME



Why build this now?

The risks around digital ownership are evolving. AI tools can help criminals create more believable messages, fraudulent sites, synthetic identities, and convincing impersonations at greater speed and scale. The FBI has specifically warned about generative AI being used for financial fraud and social engineering [1].

No wallet can prevent someone from being tricked into approving a malicious transaction. And changing a spending key does not cancel dangerous permissions already granted to another smart contract. But it can remove one persistent weakness: an older, copied spending key no longer remains a permanent means of authorizing transactions after the key has rotated.

Think of this as reducing how long a particular stolen key can remain useful, rather than promising that bad things can never happen. In a world where attempts to compromise people and devices are becoming easier to scale, reducing long-lived access is a practical design choice.

What makes KNOAD different?

Conventional self-custodial wallet	KNOAD Wallet
Spending authorization is typically bound to a long-lived key.	A fresh spending key is installed after each qualifying included operation.
Replacing a compromised EOA key normally means moving assets to a new address.	The account address stays the same when the active spending key changes.
The user must handle the consequences of a long-lived key.	Rotation is part of the transaction workflow, not an extra task.
Recovery credentials and everyday spending authority are often the same underlying secret.	The 24-word recovery authority is separate from daily spending keys.

Other smart-account wallets can change signers. KNOAD's particular choice is to make that replacement automatic after each qualifying operation, not to claim that no other wallet can ever rotate a key.

What KNOAD does — and what it doesn't

It does: retire old spending keys; keep the account address through rotations; separate recovery authority from ordinary spending; keep signing on the phone; allow a keyless browser relay for website connections; narrow a new token grant to the exact amount and a new NFT grant to one token; tell you when an older permission is still live; let you speed up or cancel a stuck send; and link Discord without handing it a key.

It does not: make your phone immune to malware; reverse transactions an attacker already landed; erase token or NFT allowances that already exist; make a stolen 24-word phrase harmless; remove network fees; scan the chain for old approvals; or provide post-quantum cryptography.

WHAT THE OWNER GETS

Protected per chain
Each chain has its own key.

Exact ERC-20
Not an unlimited allowance.

One NFT
Not the whole collection.

Leftover perms
Called out. Not erased.

Stuck send
Speed up or cancel.

Discord link
No key. No gas.

THE BOUNDARY OF THE CLAIM

Ends
A retired spending key cannot authorize the next operation on that chain.

Still exposed
The current key, a compromised phone, and approvals already granted.

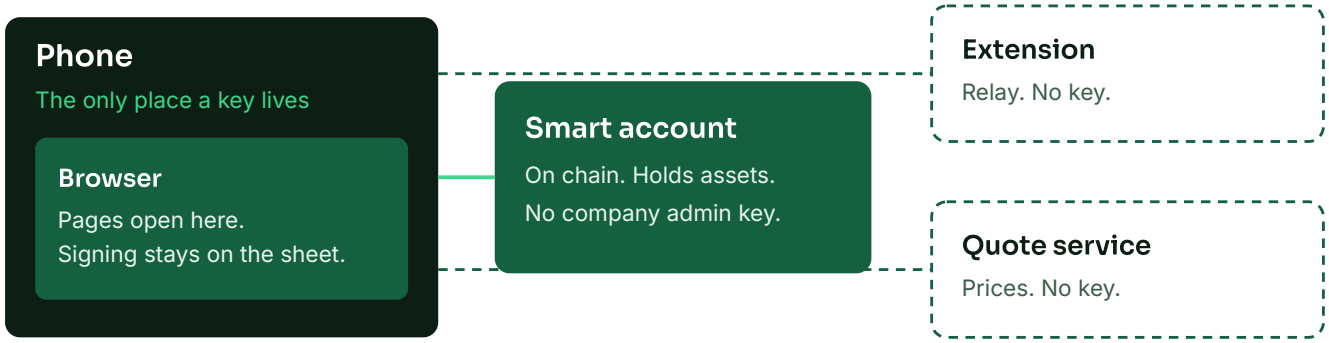
Out of scope
A stolen recovery phrase, contract defects, and quantum attacks.

Part II — How it works

1. The system at a glance

KNOAD deliberately separates the part that holds assets from the part that signs today. A smart account controls assets and records which spending key is currently allowed to authorize transactions. The phone creates and holds signing keys, prepares transactions, and presents approval screens. A browser extension relays website sessions to the phone but never holds the wallet's keys. An optional quote service fetches swap and bridge prices; it has no authority to sign or recover the account.

WHO HOLDS WHAT / WHERE TRUST STOPS



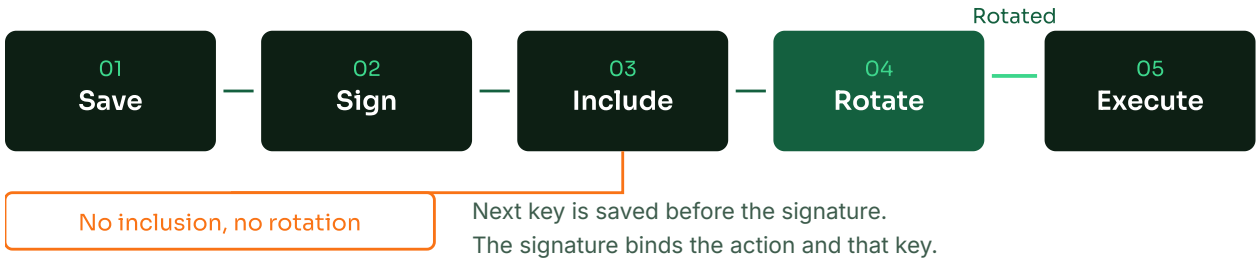
The account and factory hold the assets and do not store the phrase. They have no Cronosynthesis administrator key. The extension can learn the address and cannot sign. The quote service is untrusted; the phone checks the route before it will sign. The account uses ERC-4337 through EntryPoint v0.9 for spending. Recovery is a separate direct onchain transaction. Rotating on one chain does not retire the key on another.

Also on the phone: an ENS name resolves when you send, NFTs held on Ethereum and Base can be read, and biometric unlock opens the device vault without being the 24 words. A site pairs through WalletConnect. The extension only relays that pairing. Pages opened in the phone's browser still sign on the phone's approval sheet, so the browser is not a fifth party. Discord links from Settings through knoad.io. That link cannot sign or recover.

2. The transaction that changes the key

Here is the engine of the wallet. The order matters because a failure in one stage must not be mistaken for success in another.

A SINGLE OPERATION / FIVE DISTINCT STAGES



A bundler's acknowledgment is not inclusion. While the operation is pending, the old key is still current.

Event	Has the spending key rotated?	Did the requested payment succeed?
Signed, but not included	No	No onchain execution yet
Included; validation failed	No	No
Included; validation succeeded; execution reverted	Yes	No
Included; validation succeeded; execution succeeded	Yes	Yes

Saying every successful payment rotates is wrong in the most pedantic, important way: a payment that fails still retires the key.

A small state model

Let K[c,g] be the currently authorized spending key on chain c at generation g. Let N be the next spending key committed by a valid signed operation, and let A be its requested calls.

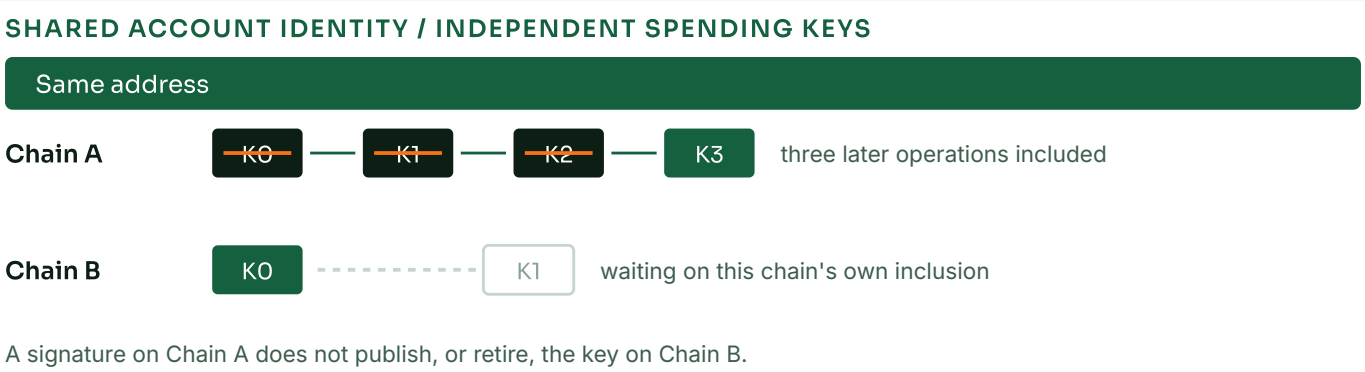
```
ValidIncludedOp(c,g,K[c,g],N,A) => K[c,g+1] := N ; generation[c] := g+1 ; then execute(A)
```

If validation fails or the operation never lands, K[c,g] remains authorized. If execution reverts after valid inclusion, the transition to K[c,g+1] still stands. This is a conceptual state equation, not contract source code.

The account uses one transaction lane. Two conflicting candidate next keys cannot be sent as independent concurrent operations with the same sequence. A pending operation may instead be replaced with a higher-fee version or cancelled using the same previously saved next key. Activity offers both. Settings can prepare the higher-fee replacement after 15 minutes, and the owner still approves it. A key refresh is an operation with an empty execution payload and still rotates; an encoded batch containing zero calls is rejected. Ordinary batches contain at most eight calls, and arbitrary self-calls into the account are restricted.

3. One wallet, separate chain state

A single recovery identity determines a stable smart-account address across the currently listed networks. That shared address is a convenience. It is not the security mechanism.



A separate initial spending key is derived for each supported chain. A newly used chain can receive assets before the account contract is deployed there. Its first key is installed only through a chain-specific activation signed by the original recovery authority.

The phone's registry names Ethereum, Base, Sepolia, and Base Sepolia. Addresses are in Appendix B. The factory, verifier, and account implementation are broadcast on all four, and the runtime code matches this tree. Recovery has been included on Sepolia and Base Sepolia. Ethereum and Base have the bytecode and

have not had a recovery included. BNB Smart Chain, Arbitrum, Robinhood Chain, Monad, and ApeChain are registered and not deployed. The phone does not advertise them.

4. Recovering the wallet — without creating a company back door

Your 24-word recovery phrase is the way to regain control if your phone is lost or the active spending key becomes unavailable. It is not the key the phone uses for everyday transactions. KNOAD does not retain the phrase and cannot recover it for you. The words are entered once and then cleared. They are not saved. Secure all networks is optional. Anyone can pay a chain's fee. The spending key cannot take recovery, the verifier, or an upgrade for itself.



On a replacement phone, the user enters the phrase, derives the recovery authority, and can locate the wallet and inspect readable chains without an onchain transaction. The phone prepares chain-specific recovery steps. Each chain must accept its recovery steps in order, and the previous phone can still spend there until that chain actually includes the switch.

Recovery is a direct transaction: a separate account can pay the network fee, even if the KNOAD account has no ETH. A chain that has not yet deployed the account may require special first-use handling. Whoever lands a valid first use or recovery first controls the immediate result; a stolen device can race the owner. This is a real timing risk, not a footnote to hide.

Situation	Designed response	Remaining risk
Phone lost, phrase safe	Default instant recovery installs a new spending key on each recovered chain.	Old phone remains capable until each chain accepts its switch.
Phrase stolen, phone safe	Owner may optionally configure a recovery delay and current-key veto.	The thief can still attempt recovery; without a configured delay, exposure is urgent.
Phrase and phone stolen	Recovery can be contested.	A delay plus veto can create a deadlock.
Upgrade wanted	Recovery-signed, codehash-checked upgrade with a minimum seven-day wait.	Malicious recovery authority still matters; not an instant defense.

The default recovery delay is off, because a mandatory wait can make a lost-phone emergency worse. Tightening spending policy can apply immediately with recovery authority; loosening policy is subject to the configured wait and veto process. The everyday spending key cannot replace the recovery authority, change the verifier, or schedule an implementation upgrade.

5. What key rotation does not revoke

Rotation changes your locks. It does not take back the spare key you already handed the neighbor.

WHAT ROTATION REVOKES / WHAT PERSISTS

Ends	Stays until revoked elsewhere
Old spending key. No new authorization.	Token allowance, in the token contract.
Login signature. The current-key check fails.	NFT operator, in the NFT contract.

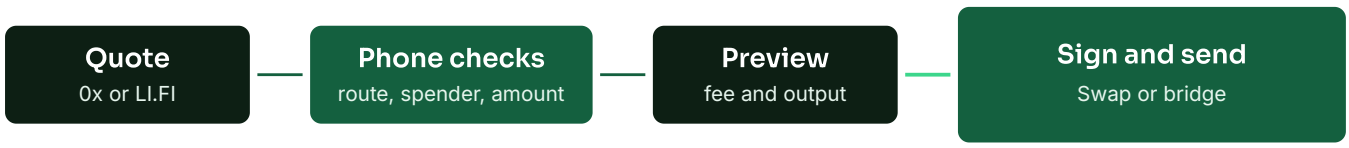
- An approved spender may still move funds without a new KNOAD signature. Disconnecting a website does not remove that, and reading an NFT on the home screen does not either.
- An ordinary login signature is checked against the current key and generally stops validating after rotation under the documented ERC-7739 flow.
- An explicit marketplace order grant can remain valid across rotations if it was registered with a named caller and an expiry. Recovery ends the grant by advancing the security epoch. Marketplace acceptance has not yet been demonstrated.

Rotation still does not erase a grant that already lives in a token or NFT contract. The phone says so in Security Center, and again after a rotation, a disconnect, or a new grant. It does not scan those contracts or revoke them for you. A new site grant is narrowed before it is signed. An unlimited ERC-20 approve becomes the exact amount spent in that same transaction, or the phone refuses it. A collection-wide NFT operator grant becomes approval of one token id when that id is in the batch or is the only one of that contract held on the phone, or the phone refuses it. Sending an NFT from the home screen already moves that one token. The account's creation policy still allows a maximum approve, because turning that switch on at creation would change the address. The phone's default does not.

6. The optional trading layer

The wallet's core and the optional commerce features have different dependencies. Creating, receiving, sending, recovering, rotating, managing permissions, and pairing a site do not incur a KNOAD service fee. Normal network gas still applies. Swaps and bridges may carry a clearly displayed KNOAD fee.

OPTIONAL SWAPS / BRIDGES / VERIFICATION BEFORE SIGNING



A same-chain swap obtains candidate routes through 0x providers; a bridge uses LI.FI. The phone does not blindly trust the quote server. It verifies approved destinations, exact token approvals, intended receiver, token amounts, minimum output where readable, slippage, and route structure. For 0x, the Settler is checked against the provider's onchain registry; Permit2 and permit-style routes are refused. The phone rejects routes it cannot recognize and prechecks optional spending policy.

Commerce rule	Current documented behavior
KNOAD fee	0.25% default on an optional swap or bridge; capped at 0.30%
No configured recipient	KNOAD fee is zero
Comparison	Prefer the higher minimum received; when within 0.1%, lower network fee wins

Commerce rule	Current documented behavior
Quote validity	Refetch after 30 seconds or the provider deadline; signed calls must match displayed calls
Approvals	Exact input amount to the intended provider; no unlimited commerce approval

Live swap and bridge submit after those checks. Commerce follows the same exact-approve rule: the allowance is the amount being sold, not an unlimited grant. Some LI.FI destination-side minimum or receiver details are still not fully readable from shared bridge data. Buy / Sell and listings stay inactive. The home screen can read NFTs held on Ethereum and Base. That read is not a trade.

7. If Cronosynthesis disappeared

If Cronosynthesis disappeared, your wallet would be the only thing in the building still working.

KNOAD is designed so the company does not have an administrator key to freeze an account, replace its signer, or upgrade its code. The published factory and recovery design make address computation independent of a KNOAD index. Another bundler can submit a signed operation; a recovery can be submitted by anyone willing to pay its gas.

Linking Discord is a perk, not a key. From Settings, Continue with Discord returns through knoad.io. The phone signs a verification message: no gas, and no token approval. A participating server can then recognize the wallet for a role. The link cannot sign, recover, or upgrade. If it disappeared with the company, the account on chain would be unchanged.

The company-run quote service is an optional dependency: if it vanished, swap and bridge pricing would stop, while basic sends, recovery, and phone-mediated pairing are designed to remain possible.

8. What this design protects against — and what it doesn't

The boundary drawn in Part I is the whole claim. Reduced risk is an old key after its retirement. Still exposed are the current key, a compromised phone, and approvals already granted. Out of scope are a stolen recovery phrase, contract defects, and quantum attacks.

AI makes the social layer of security harder, not easier. KNOAD does not pretend otherwise. A convincing phishing prompt can still trick a person into signing a harmful current transaction. A compromised phone can steal while it controls the current key. A stolen recovery phrase can reclaim the account. Previously granted allowances can outlive all rotations.

Where the design helps is narrower and meaningful: once a chain processes the valid transition, a previous spending key is no longer a standing authorization for future spending on that chain. The phone also narrows the next grant a site can receive: the exact token amount in that transaction, or one NFT, and it refuses the unlimited and collection-wide versions. That's a reduction in persistent exposure, not a substitute for device hygiene, permission management, recovery-phrase protection, contract review, or careful signing.

Quantum reality check

Rotation is many things. Post-quantum is not one of them.

KNOAD presently uses conventional elliptic-curve cryptography (secp256k1 and P-256). Key rotation is not post-quantum cryptography. NIST identifies elliptic-curve systems among the cryptographic systems threatened by a sufficiently capable quantum computer [2]. KNOAD's recovery-controlled verifier upgrade path leaves a potential route for future algorithm changes, but it does not mean a quantum-resistant verifier has been built, audited, or shipped.

Part III — Engineering reference

9. Authorizations, derivation, and address identity

The account's address is independent of the active hot signing key. The account is created through a deterministic factory using the original recovery authority, account index, and hash of the initial creation policy. In conceptual terms:

```
SALT = H(originalRecoveryAddress, accountIndex, H(initialPolicy))
ACCOUNT = CREATE2(factory, SALT, H(proxyInitCode))
```

The actual address also depends on the published factory, implementation, and proxy initialization construction. These lines describe the dependency structure, not a byte-for-byte drop-in address calculator. The first active spending key is deliberately absent from the address derivation.

Secret / use	Derivation or behavior
Human backup	24-word BIP-39 mnemonic with an empty BIP-39 passphrase in this build
Recovery at epoch e, account a	m/7331'/60'/a'/0'/e'
Initial chain key, account a, chain chainId	m/7331'/60'/a'/1'/<chainId>'
Later spending keys	Fresh random device-bound keys where hardware support allows; not derived from the mnemonic
Device app password	Unlocks and encrypts the phone vault, including by biometric unlock where the device offers it. It is not recovery material.

The one-time initial hot key uses secp256k1. Later device-bound keys may use Secure Enclave or StrongBox facilities when supported and may be nonexportable. The phone uses account index zero in the source implementation. A BIP-39 passphrase cannot be added later without changing the derived account; the current UI does not collect one.

Recovery epochs are read from chain state. The device does not need to guess or scan recovery indices. The first-use activation is signed for a specific chain and becomes unusable once that chain's account is deployed. If a compatible account already exists, the factory returns it without replacing its active key.

10. Contract behavior by authority

Operation	Who can authorize it?	Result / restriction
executeAndRotate	Current hot signer through EntryPoint	Valid inclusion installs the next signer before permitted calls execute.
Empty-payload key refresh	Current hot signer through EntryPoint	Rotates without executing calls.
vetoRecovery	Current hot signer	Vetoes a pending delayed recovery or loosening, and rotates.
recover	Recovery authority	Replaces the hot signer immediately or schedules a delayed replacement; replay-protected.
setPolicy	Recovery authority	Tightening is immediate; loosening is subject to delay and veto.
approveOrderDigest / revokeOrderDigest	Permitted account self-call path	Register or revoke one caller-bound ERC-1271 order digest.

Operation	Who can authorize it?	Result / restriction
scheduleUpgrade	Recovery authority	Schedules a codehash-bound implementation change after at least 7 days.
cancelUpgrade	Recovery authority	Cancels a pending change with an authorized signature.
executeUpgrade	Pending scheduled path	Rechecks the implementation codehash.

Generic execute and arbitrary account self-calls are restricted; initialization directly on the implementation reverts. The contract accepts standard ERC-721 and ERC-1155 receiver hooks. The account has one sequence lane; the phone must preserve the saved next key when speeding up or cancelling a pending operation. These are the intended behaviors described in the source paper, not an independent audit finding.

```
phone:
  nextKey = generate_and_save(expectedGeneration + 1)
  userOp = bind(requestedCalls, nextKey, expectedGeneration, chainId)
  signature = sign(currentKey, userOp)
  submit(userOp, signature)

account validation (if included):
  require(signature matches currentKey)
  require(generation/nonce matches)
  require(nextKey is permitted and nonzero)
  currentKey = nextKey
  generation += 1

execution:
  run requestedCalls   # may revert after rotation
```

This is illustrative pseudocode. It omits ERC-4337 calldata encoding, signature domains, transaction gas accounting, internal operation binding, and contract-specific revert paths. Consult Spec 1.4 and the source code for authoritative implementation behavior.

11. Gas and execution evidence

Local Foundry/Forge runs, compiled with Solidity 0.8.28 and optimizer 200 against EntryPoint v0.9. The figures include test harness overhead and are not mainnet gas quotes.

LOCAL FORGE GAS MEASUREMENTS / NOT MAINNET QUOTES

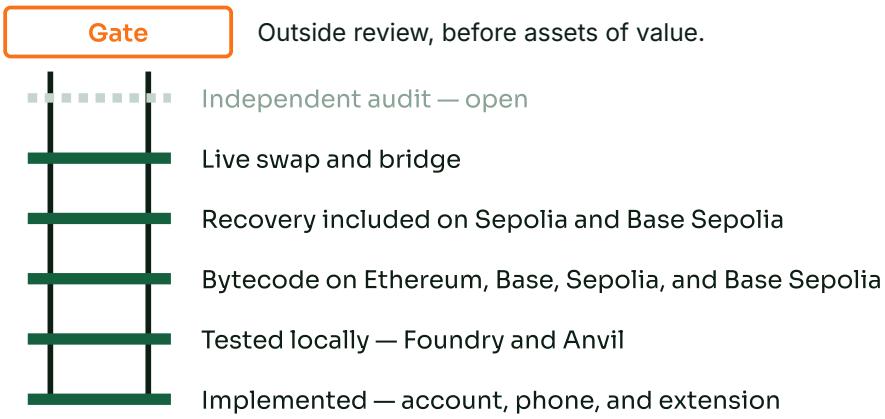
Create account	Send ETH and rotate	Direct recovery
223,447	194,768	28,417

Compiler: solc 0.8.28. Optimizer: 200. EntryPoint: v0.9. Test harness included. Network gas prices, sponsorship, deployment, and calldata will change what a person actually pays. There is no paymaster in this version. A recovery can be paid by any outside account, but that does not mean KNOAD pays the fee.

12. Evidence, limitations, and release gates

Ladders are only useful if you admit which rungs are missing.

EVIDENCE IS A LADDER, NOT A SINGLE CHECKMARK



Evidence the ladder does not show	Status as of 21 October 2026
Local Foundry suite	Passing without a fork URL.
P-256 check	Forge check. Excluded from the cited Anvil scenario because its precompile was unavailable there.
Recorded Foundry fork of a live swap	Not in the cited local suite. Swap and bridge run on the phone.
Commerce transaction submission	On. A checked swap or bridge is signed and sent.

Release posture: this design may be promising, but project-authored tests are not an external security audit. Contracts described here are not cleared to hold assets of value. Marketing should not imply otherwise.

13. Threat register — representative failure modes

Threat	Designed mitigation	Residual exposure
An old spending key leaks	Included key rotation makes it invalid on that chain.	The current key may be exploited before the next inclusion. Other chains are independent.
Phone compromise	Device-bound keys where available; off-page phone approval; recovery authority is separate.	Malware controlling the live phone can still authorize theft, and may follow future keys.
Phrase compromised	The owner may enable delayed recovery and a hot-key veto.	The phrase controls recovery. Instant default recovery leaves no automatic veto.
Pending operation stuck	Replace or cancel at the same nonce using the saved next signer.	No inclusion means no rotation. Censorship can prevent inclusion.
Token or NFT permission abuse	The phone narrows a new unlimited token approve to the amount in that transaction, and a collection grant to one token id, or it refuses the request. Commerce uses the exact sell amount.	Old approvals remain. The phone does not scan or revoke them. An unfamiliar spend function can still get through.
Key update interrupted by a crash	Persist the next signer before signing.	Lost local state may still require phrase recovery.
First use on an undeployed chain	Chain-bound recovery authorization is required for the initial signer.	Whoever lands a valid activation or recovery first can win. A token-only chain may need outside gas.

Threat	Designed mitigation	Residual exposure
Quote service compromised	The phone independently validates pinned contracts, the route, the spender, and the amount.	Bad pricing or service loss remains possible. Bridge destination checks are incomplete.
Malicious implementation update	Only recovery authorization, a wait of at least 7 days, and a codehash recheck.	Stolen recovery authority can schedule an upgrade. There is no company pause key.
Future cryptographic breakthrough	A recovery-controlled verifier upgrade path may support migration.	Current signatures are not quantum resistant, and migration is not implemented.

14. Continuity and dependency model

A self-custodial architecture does not guarantee uninterrupted RPC access, bundler participation, or correct third-party tooling. An unrecoverable bug in the recovery path could defeat the model. There is no company-wide emergency pause switch.

Appendices

Appendix A — Plain-English glossary

Term	Meaning
Smart account	An onchain contract account that holds assets and checks authorization rules.
Spending key / hot signer	The currently authorized key used for everyday operations on one chain.
Recovery authority	The separate owner-controlled identity derived from the 24-word phrase.
Rotation	Replacing the active spending key and retiring the previous one.
Generation	The account's transaction sequence for hot operations.
Inclusion	The chain accepted the operation. Not the same as a bundler receiving it.
Validation	The account and EntryPoint accept its authorization and nonce.
Execution	The requested calls run. They may fail even if validation and rotation succeed.
Bundler	An ERC-4337 participant that packages UserOperations for inclusion.
Approval	Permission recorded in a token or NFT contract. Key rotation does not revoke it.
Quote	A proposed trade or bridge route checked before any signature or submission.

Appendix B — Supported networks and published development values

Not cleared for assets of value. Addresses as broadcast on 21 October 2026.

Item	Documented value
Ethereum	Chain ID 1. Broadcast. Recovery not included.
Base	Chain ID 8453. Broadcast. Recovery not included.
Sepolia	Chain ID 11155111. Broadcast. Recovery included.
Base Sepolia	Chain ID 84532. Broadcast. Recovery included.
BNB Smart Chain	Chain ID 56. Registered, not deployed. Not on the phone.

Item	Documented value
Arbitrum	Chain ID 42161. Registered, not deployed. Not on the phone.
Robinhood Chain	Chain ID 4663. Registered, not deployed. Not on the phone.
Monad	Chain ID 143. Registered, not deployed. Not on the phone.
ApeChain	Chain ID 33139. Registered, not deployed. Not on the phone.
EntryPoint v0.9	`0x433709009B8330FDa32311DF1C2AFA402eD8D009`
Recovery verifier	`0x39305881423DC27C6238bE3A343787772534e764`
Factory	`0x640b63604581cC0b1326025a57C188BCc6160Dc9`
Implementation	`0x70044945917051ce6DcCD244C4d4288FE14A9f28`

Published route checks in the source: 0x AllowanceHolder `0x0000000000001fF3684f28c67538d4D072C22734`; LI.FI Diamond `0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE`; 0x registry `0x0000000000004533Fe15556B1E086BB1A72cEae`. The Settler is checked dynamically, not permanently pinned. Permit2 `0x000000000022D473030F116dDEE9F6B43aC78BA3` is rejected in this documented commerce path.

Sources and document status

Technical source: KNOAD Wallet technical white paper, Cronosynthesis, 16 October 2026. This narrative edition was revised on 21 October 2026. Spec 1.4 is the normative contract text.

External context (not evidence that KNOAD is secure):

[1] FBI Internet Crime Complaint Center, Criminals Use Generative Artificial Intelligence to Facilitate Financial Fraud, 3 December 2024. https://www.ic3.gov/PSA/2024/PSA241203

[2] NIST National Cybersecurity Center of Excellence, Frequently Asked Questions about Post-Quantum Cryptography. https://pages.nist.gov/nccoe-migration-post-quantum-cryptography/